

Submit VI

Oscar Vargas Pabon

Las implementaciones estan en https://github.com/osvarp/paralelComputing_week3

Bajo la carpeta /submit6

Minimum and maximum

Use un reduction de omp para calcularlo, en pseudocodigo seria

```
Parallel for i<-0 to vec.size() do
```

```
    mn <- min(mn,vec[i])
```

```
end for
```

parallel prefix sums

Use un metodo que encuentre en https://en.wikipedia.org/wiki/Prefix_sum

Hace $O(n \lg n)$ de trabajo, pero puede paralelizarse hasta $O(\lg n)$

En pseudo código seria

```
For e<-0 to log(vec.size()) do
```

```
    Parallel for i<-0 to vec.size() do
```

```
        If  $j < (1 \ll e)$  do
```

```
             $\text{Res}^{e+1}_j \leftarrow \text{Res}^e_j$ 
```

```
        Else do
```

```
             $\text{Res}^{e+1}_j \leftarrow \text{Res}^e_i + \text{Res}^e_{i-(1 \ll e)}$ 
```

```
        End if
```

```
    End for
```

```
End for
```

Sorting

Bubble sort lo implemente como se presento en clase

Merge sort esta melo

Quick sort lo implemente sin memoria auxiliar y para que funcione en $O(n \lg n)$ esperado.

Las versiones paralelas de mergeSort y quickSort se realizaron paralelizando los subproblemas que se generan.